Vendor onboarding

From submission to approval

Project report | Zamp 2.0 | Aditya Jain

Problem statement

Before paying a vendor, a company needs complete and consistent identity, tax, business and bank information. The supplied PS-2 highlights incomplete submissions, wrong attachments, mismatched account-holder names, malformed tax identifiers and subtle cross-field inconsistencies. Manual review and follow-up also leave a fragmented audit trail. [1]

The required outcome is a working process that accepts a vendor submission, returns **Approved, Pending or Rejected** with visible reasoning, and explains what is needed for anything not approved. The brief allows the builder to define the submission format, rules and output, with realistic edge cases. [1]

Delivered approach

The host-company operator uploads vendor documents directly. The application reads or transcribes them, extracts source-linked fields, matches PAN/GSTIN to the vendor master, applies Python validation rules, and stores a versioned result with a follow-up message. One request can contain a combined PDF or several supported files.

Expectation	Delivered implementation
Review real inputs	PDF, PNG, JPEG and DOCX intake, including scans and combined evidence.
Make reasoning visible	Individual checks, source quotes, master comparison and conflicting values.
Communicate next steps	Editable on-screen follow-up; no email-delivery dependency.
Handle exceptions	Missing evidence, identity conflicts, changed bank details and linked corrections.
Preserve history	Stored uploads, review attempts, processing events and reference snapshots.

Decision policy

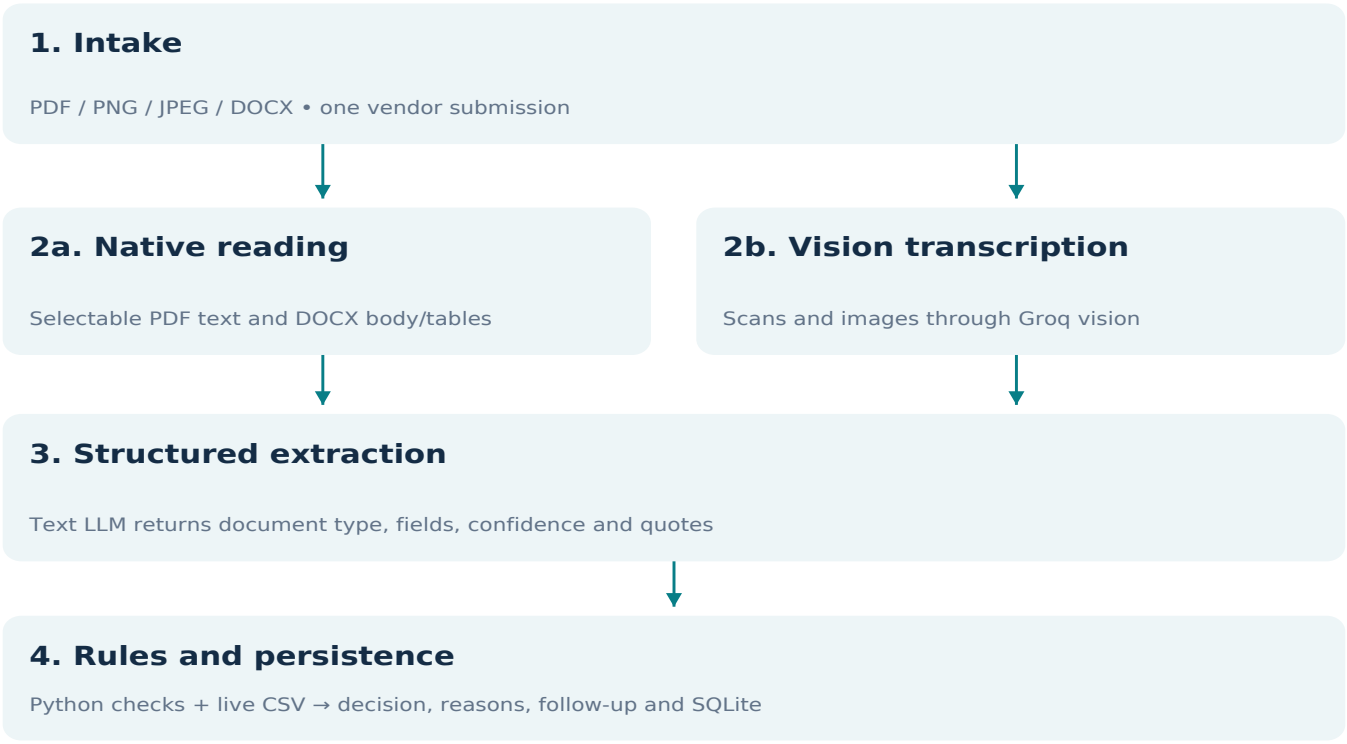
Status	Meaning
Approved	Required checks pass against one existing approved vendor record.
Pending	An issue remains unresolved, or a new vendor needs procurement verification.
Rejected	Reliable identifiers match an internally blocked vendor record.

Report guide: architecture/models p. 2; checks p. 3; databases p. 4; dataset p. 5; UI/demo pp. 6-8; engineering p. 9; implementation/run guide p. 10. Automated checks assess completeness and internal consistency, not government authenticity.

01 / PROJECT REPORT

Architecture and model roles

AI reads and structures the evidence; explicit Python rules determine the business decision.



Role	Configured model	Responsibility
Text	openai/gpt-oss-120b	Classify evidence, extract quoted fields and draft issue-specific follow-ups.
Vision	qwen/qwen3.8-27b	Transcribe scans, images and DOCX embedded document images.

Evidence control

Native PDF and DOCX text enters extraction directly. Images and scans first pass through vision transcription. Every extracted field includes a value, source quote, confidence and file/page or section locator. The application checks the schema and verifies source support before using the value.

Unsupported fields, uncertain classification or reading failures create review issues. The application does not fill missing evidence from the master and does not let a model choose the approval status. Groq hosts inference, so the local application does not require a GPU.

Execution and resilience

The pipeline records reading, classification, extraction, checking, decision and drafting events. Queued work is claimed before processing; UI callbacks show live progress and saved events form the timeline. Follow-up drafting has a fallback if a response fails or is incomplete.

Model IDs are configured in .env; account access and quotas apply. Document text is treated as untrusted evidence rather than application instructions. Confidence scores are model-provided indicators, not authenticity probabilities. [2,3]

02 / PROJECT REPORT

Evidence and validation rules

The India-focused review covers identity, tax registration, banking and business-registration evidence.

Evidence	Required or checked fields
PAN proof	Legal name and PAN.
GST registration	Legal name and GSTIN; compare the PAN embedded in GSTIN.
Bank proof	Account holder, account number and IFSC.
Business registration	Legal name and registration number.
Other extracted details	Address is compared when present; trading/contact details are optional.

Companies require PAN, bank proof and business registration. Individuals and sole proprietors require PAN and bank proof. GST evidence is required when present in the submission or the matched master. A new vendor without it needs an explicit written non-registration explanation. An existing approved master record without a GSTIN can pass if its other checks pass.

Parameter	Implemented check
PAN	Five letters, four digits, one final letter; holder category consistent with the vendor route.
GSTIN	15-character structural pattern; its embedded PAN must agree with PAN evidence.
IFSC / account	Four letters + 0 + six alphanumeric characters; account number is 9-18 digits.
Confidence / quotes	Document ≥ 0.65; field ≥ 0.60; finite scores in 0-1 and source-supported quotes.
Completeness	Required document types and reliable fields must be present and readable.
Consistency	Compare legal name, PAN, GSTIN, address, account holder and bank details across evidence.
Master differences	Changed identity, address or bank information requires separate verification.
Account-holder relation	Agree with the legal name or an established relationship in the matched master.
Status precedence	A blocked-record rejection wins; otherwise any unresolved check keeps Pending.

Matching and normalization

Reliable PAN/GSTIN values identify candidate master records. Name-only matching is not used. Multiple records require identity review; missing or invalid identifiers are distinguished from a completed search with no match. Case, punctuation and spacing are normalized, with common company-suffix variants handled.

Profiles use document headings, labels and required fields, not fixed reference-PDF appearance. Address comparison is not semantic address resolution. Registration numbers are checked for readable presence, not authenticity. There is no Aadhaar check, GSTIN checksum verification or live government/bank validation.

03 / PROJECT REPORT

Reference data and review storage

The live vendor master supplies comparison values; SQLite preserves what happened during each review.

Store	Purpose and default location
Vendor master	data/vendor_master.csv: current identity, tax, banking, address, status and master-version values.
Review database	data/runtime/onboarding.sqlite3: submissions, attempts, checks, evidence, messages and events.
Uploaded copies	data/runtime/uploads/: original files grouped by submission ID and saved with unique names.

The supplied reference records

ID	Synthetic vendor	Status
V100	Northstar Office Supplies Private Limited	Approved
V101	Meera Rao	Approved
V102	Blocked Demo Trading	Blocked
V103	Seabrook Instruments Private Limited	Approved
V104	Rohan Shah	Approved

The master is read fresh for each processing run. CSV/XLSX imports validate and replace the reviewed master; exporting a spreadsheet does not make the app use it automatically. A new vendor is not automatically added or approved. Master-record version and review-attempt number are separate concepts.

SQLite tables and traceability

Table	Recorded information
submissions	Request ID, timestamps and latest attempt number.
versions	Stage, decision, evidence, checks, follow-up, errors and master/profile/policy snapshots.
files	Original/stored names, type, size, SHA-256 and submission/version links.
events	Processing-stage messages and saved reviewer actions.

Foreign keys, transactions and stale-version guards protect linked updates. Originals and earlier decisions remain available after a correction. Saved snapshots explain the reference context used at the time; later CSV or code changes do not silently rewrite history.

Back up the complete runtime directory while the app is stopped. Regenerating demo files does not replace runtime copies. Hashes aid traceability, not authenticity. The application has no login or application-level storage encryption; normal-view bank masking does not encrypt stored evidence.

04 / PROJECT REPORT

Dataset design and scenarios

Controlled fixtures test business conditions, input formats and correction behavior independently of model output.

cases.json defines values, evidence types/formats and deliberate overrides. make_dataset.py validates and packages them; document_designs.py draws the layouts; check_dataset.py checks files, expected text and page bounds. Outputs include case folders, a manifest and XLSX copies. The background watermark is removed; a small demo footer remains.

Case	Scenario	Expected result
C01	Complete new company	Pending: procurement
C02	Existing approved company	Approved
C03	All evidence in one PDF	Pending: new vendor
C04	Mixed images and PDF	Pending: new vendor
C05	Missing bank proof	Pending: missing evidence
C06	Harmless company-suffix variation	Pending: new vendor
C07	PAN differs from PAN in GSTIN	Pending: identity conflict
C08	Conflicting legal names	Pending: name conflict
C09	Existing vendor changes bank details	Pending: verify change
C10	Blocked identifier match	Rejected
C11	Unreadable bank image	Pending: clearer evidence
C12	Invoice instead of bank proof	Pending: correct evidence
C13	Malformed PAN and GSTIN	Pending: corrected IDs
C14	Existing proprietor without GSTIN	Approved
C15	New proprietor lacks GST explanation	Pending: explanation/review
C16	Existing company; no manual vendor ID	Approved
C17	Bank proof image embedded in DOCX	Pending: new vendor
C18	Correction supplies missing bank proof	Pending, then procurement
C19	Registration in DOCX table	Pending: new vendor
C20	Scanned combined PDF	Pending: new vendor

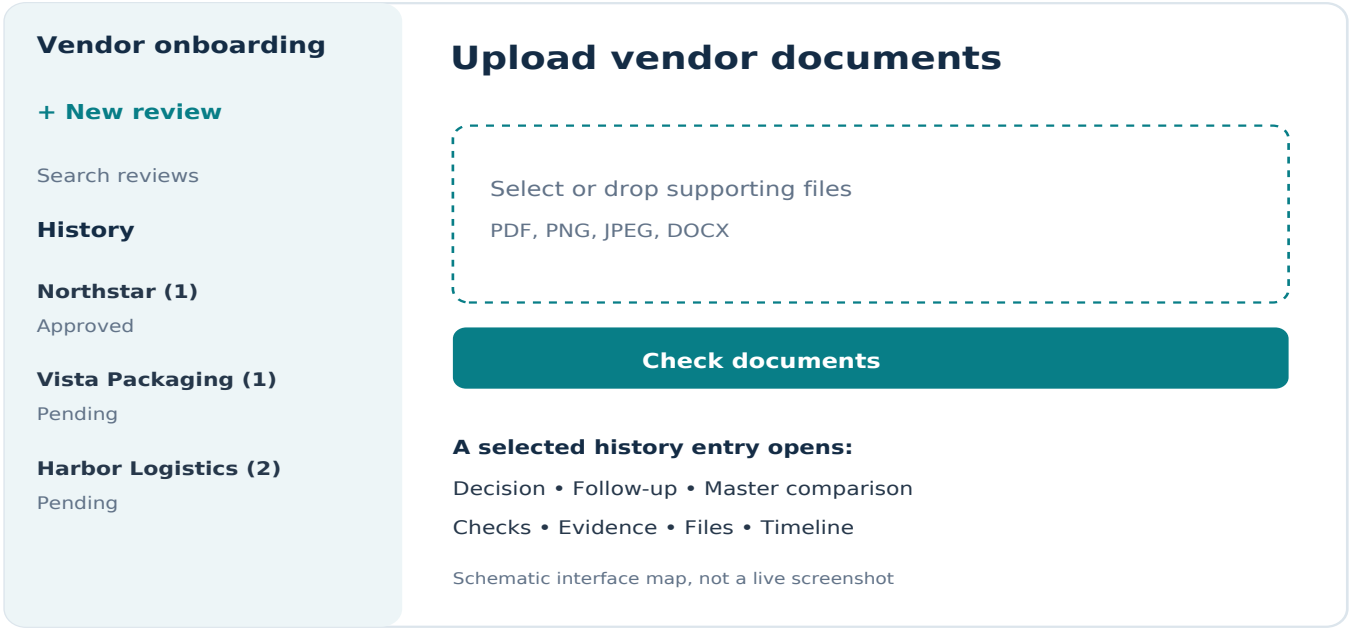
These are 20 submission scenarios, not 20 registered vendors. expected_results.json is an offline answer key; the running app never reads it to assign decisions. Live extraction can introduce extra review issues. No manual form.json entry or reference-PDF comparison is required.

Generation uses staged replacement and explicit --overwrite handling. The checker validates files/layouts, not live OCR accuracy. Combined PDFs, raster images and DOCX variants exercise the same downstream rules. C02, C14 and C16 demonstrate approval; C10 demonstrates rejection.

05 / PROJECT REPORT

Streamlit features

Direct upload and sidebar history keep the main workflow accessible while deeper evidence remains inspectable.



Feature	How it helps the reviewer
Upload homepage	Submit combined/multiple documents without a manual identity form or extra creation page.
Sidebar history	Company (attempt), status icons/captions, search, filtering and direct result navigation.
Live progress	Reading, extraction, checking, decision and drafting stages appear as processing runs.
Decision and follow-up	Status and missing evidence appear first; the latest completed message can be edited/saved.
Master comparison	Documents beside saved reference values; conflicting candidates show source files/pages.
Evidence and checks	Document type, confidence, profile result, source quote, next action and provenance.
Previews/downloads	PDF page selection, image display, DOCX text/embedded images and original download.
Correction and timeline	Linked versions, explicit replacement, earlier outcomes and saved processing events.
Bank display controls	Masked summary values; a separate reviewer expander can reveal the full extracted account.

The interface map is a labeled schematic, not a live screenshot. History covers attempts in the 200 most recently updated submissions. A separate New review creates another ID, so a repeated company name can appear as another (1). Text/container rendering avoids the Pandas dataframe UI path.

06 / PROJECT REPORT

Demonstration: decisions and reasons

Show an approved existing vendor, a clean new vendor and an internally inconsistent submission.

Demo	Action and expected behavior
C02 / Existing vendor	Upload all initial C02 files. PAN/GSTIN matches Northstar (V100); consistent required evidence produces Approved.
C01 / New vendor	Upload the complete new-vendor set. Clean documents still leave Pending until procurement verification.
C07 / Tax conflict	Upload Vista Packaging evidence. Different PAN values in PAN/GSTIN evidence produce Pending and a correction request.

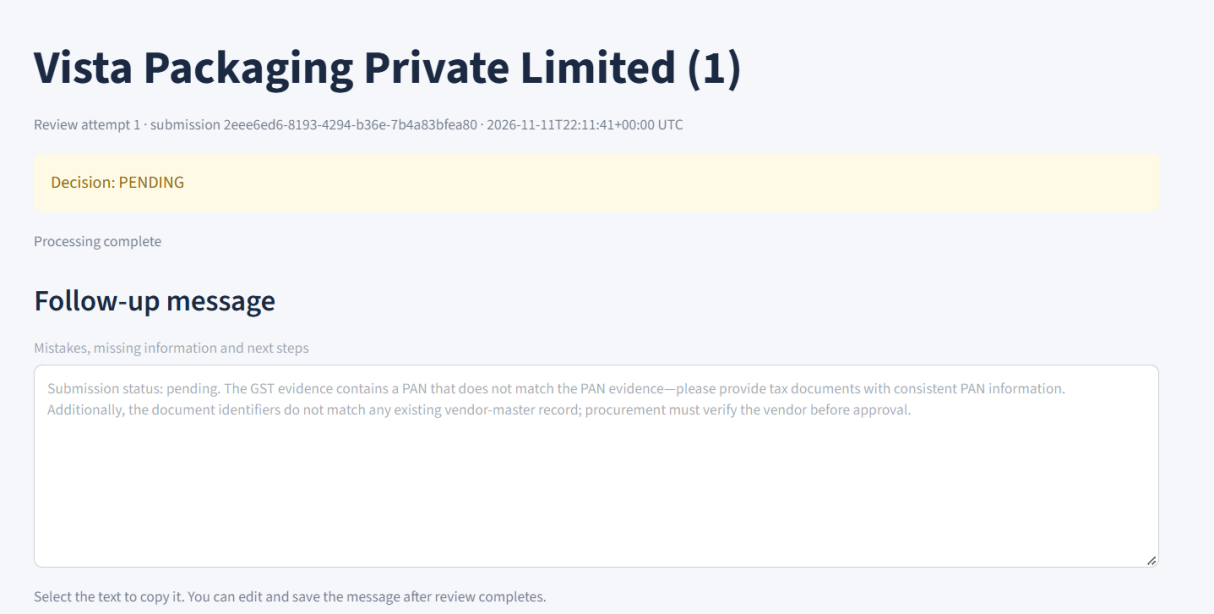


Figure 1. Actual development capture of C07: the message identifies disagreement between PAN evidence and the PAN embedded in GSTIN. Captured wording can differ from the final release.

The clean-new-vendor follow-up

When procurement verification is the **only** remaining issue, the app uses:

> **Pending: procurement verification**
>
> All required documents have passed the automated checks. No matching vendor record was found in our database, so this submission is being treated as a new vendor. Final approval is pending an in-person meeting and verification by the procurement team.

This message is withheld while missing, invalid, conflicting or unreadable evidence remains. The three demonstrations describe intended outcomes with supported extraction; they are not a claim of measured live model accuracy. The app requests correction rather than declaring that a mismatch proves fraud.

07 / PROJECT REPORT

Demonstration: correction and history

A clear next action and a linked second attempt turn an incomplete submission into a manageable review process.

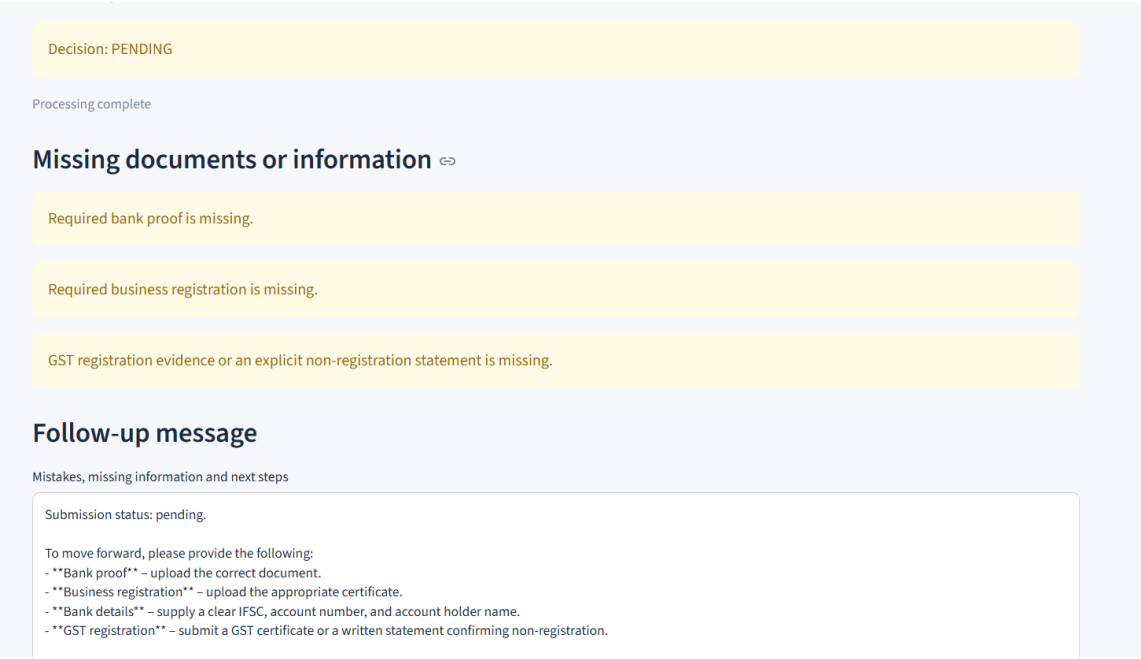


Figure 2. Actual development capture: missing bank proof, business registration and GST evidence/explanation are listed above the follow-up. Captured wording can differ from the final release.

Step	Operator action and application behavior
1. Inspect	Read the decision checks, missing-document list, extracted values and source preview.
2. Correct	Choose Upload a correction / new version on the latest result; add missing documents.
3. Replace explicitly	Select earlier wrong/unreadable files for replacement. Adding a file alone does not hide conflicts.
4. Review again	Create the next attempt under the same submission ID. Earlier originals and decisions remain visible.
5. Resolve independently	Missing bank proof can clear while a separate new-vendor verification requirement remains Pending.

Source quotes and the side-by-side master comparison help distinguish a document issue from an extraction issue. Conflicts show each candidate and its source instead of a misleading blank. Normal bank views are masked; original files still contain sensitive details.

To reprocess after a code change, a correction may retain existing documents and leave new uploads empty, provided active evidence remains. To use regenerated PDFs, upload the new files and explicitly replace old copies, or begin a New review.

Processing complete means the pipeline finished, not that the vendor was approved. Browser restarts/cache clearing do not delete the local database or saved uploads. Edited follow-up messages are saved only for the latest completed Pending/Rejected attempt; older messages remain read-only.

08 / PROJECT REPORT

Edge cases, engineering and verification

The design handles independent business conditions and preserves uncertainty instead of silently approving incomplete evidence.

Selected edge case	Response and design judgment
C05 / Missing bank proof	Pending with the absent evidence and bank fields listed. A plausible identity cannot replace financial evidence.
C07 / PAN-GSTIN conflict	Pending for consistent tax documents. Individual format validity is insufficient when fields disagree.
C09 / Bank-account change	Pending for separate verification. An identity match does not authorize a payment-destination change.
C18 / Partial correction	Bank-proof issues clear after correction, but procurement verification remains. Fixing one issue does not bypass another.

Implementation challenges addressed

Challenge	Resolution
Manual transcription	Document-only intake and automatic identifier matching.
Watermark contamination	Remove the generated overlay; separate recognized overlays in native text reading.
Hidden address conflicts	Show candidate values and file/page sources in the comparison view.
Unreliable model output	Validate schema, confidence and quotes; keep unsupported evidence Pending.
Correction overwrites	Explicit supersession, version guards and preservation of prior evidence.
Local Pandas DLL blocking	Use text/container components instead of dataframe rendering for the affected UI views.

Verification and its limits

The expanded isolated development run reported **117 passing checks**, including temporary address and follow-up regressions beyond the baseline repository suite. The distributed three test files can report a different count. Supplied C02 PDFs were read through the actual reader; model responses were mocked for repeatable pipeline tests.

Additional Streamlit checks covered upload-home navigation, historical statuses, linked corrections and conflict-source display with Pandas imports blocked. These results validate software behavior, not document authenticity or empirical live extraction accuracy.

No live model accuracy score, latency benchmark or cost benchmark is claimed. Format-valid fabricated data can still pass structural checks. Final identity, tax activity and bank ownership require the appropriate external/procurement verification.

09 / PROJECT REPORT

Implementation and operating guide

The root main.py launches the interface; app/ contains the reusable processing and persistence logic.

File / area	Responsibility
main.py	Upload UI, sidebar history, results, comparisons, previews, timeline and corrections.
app/upload.py	Type/signature/size checks, file storage, metadata and replacements.
app/documents.py	Native PDF/DOCX reading, watermark separation and vision transcription.
app/llm.py	Structured extraction, source validation and follow-up drafting.
app/profiles.py	Editable heading/label cues and required-field profiles.
app/rules.py	Normalization, matching, evidence checks and the final decision.
app/pipeline.py / db.py	Orchestration, stage events, SQLite history and master access; __init__.py marks the package.
data/	Master/profile CSVs, requirements.json, fixture definitions and offline expected results.
scripts/	make_dataset, document_designs, check_dataset, import_vendors and both CSV seed scripts.
tests/ / root setup	Rule/profile/workflow tests; requirements, environment template, Streamlit config, Docker and ignore files.

Run locally from the project root

```
python -m pip install -r requirements.txt
python scripts/make_dataset.py --overwrite
python scripts/check_dataset.py
python -m pytest tests -q
python -m streamlit run main.py
```

Use the intended Python environment and configure .env first. Generation is needed only to create/replace demonstration files; --overwrite replaces existing generated assets. Keep Docker copy/launch settings aligned with root main.py. README.md contains the full setup and file reference.

Configuration and production boundaries

Defaults: 15 MiB/file, 45 MiB/new upload batch, 30 PDF pages, eight DOCX embedded images and 25,000 characters per extraction block. max_files in requirements.json controls batch count; set it to 4 for a four-file policy. The app has no login, application-level encryption, live government/bank validation or procurement-approval/master-update action. Relevant content is sent to Groq.

References: [1] Supplied PS-2 screenshot and Candidate Guide p. 4 (paraphrased). [2] Current project source, fixtures, expected_results.json and README. [3] Groq models / vision; Streamlit documentation. Figures 1-2 are supplied development captures; the interface map is a schematic. Reference vendors and scenario data are synthetic.